Vidyalankar Institute of Technology

Department of Computer Engineering

DEVOPS MINI PROJECT

Week 12 Report

Automated Grocery Inventory Dashboard

Name:	Prachi Patil
Roll No.:	23102B0038
Class:	BE - VII
Division:	B
Academic Year:	2026–27

1. Objective

The objective of Week 12 was to extend the Jenkins pipeline so that the Grocery Inventory Dashboard could be automatically packaged, containerized, published to a Docker registry, and deployed as a fresh Docker container after successful testing.

The implemented continuous deployment flow was:

Git Commit → Jenkins Checkout → Maven Test → WAR Package → Docker Build → Versioned Image → Registry Push → Fresh Container → Deployment Verification

A local Docker Registry was selected instead of Docker Hub. The registry was configured on port `5001`.

2. Technology and Environment

Component	Configuration
Programming Language	Java 21
Framework	Spring Boot 4.1.0
Build Tool	Maven
Application Packaging	WAR
Database	MySQL
Web Server	Apache Tomcat 10.1
CI/CD Tool	Jenkins
Container Platform	Docker 29.8.1
Registry	Local Docker Registry
Registry Port	5001
Application Port	8083
Versioning	Jenkins BUILD_NUMBER
Source Control	Git / GitHub

3. Jenkins Pipeline Implementation

The Jenkins pipeline was configured using the project's `Jenkinsfile`. The pipeline contains the following stages:

1. **Checkout** – retrieves the latest project code from the Week 12 Git branch.
2. **Test** – executes the complete Maven test suite using `./mvnw clean test`.
3. **Package** – generates the deployable WAR file using Maven.
4. **Docker Build** – builds a Docker image containing the application WAR.
5. **Push Image** – pushes the versioned image to the local Docker Registry.

6. **Deploy Container** – stops and removes the previous container and starts a new container using the newly created image.
7. **Verify Deployment** – checks container status, displays container logs, and verifies the registry tag.

The Docker image is versioned automatically using the Jenkins build number:

```
localhost:5001/grocery-inventory-dashboard:${BUILD_NUMBER}
```

For the successful Week 12 pipeline execution, Jenkins generated:

```
localhost:5001/grocery-inventory-dashboard:3
```

4. Docker Image Build and Registry

After successful testing and WAR packaging, Jenkins executed the Docker build stage. The generated WAR file was approximately 51 MB.

```
./mvnw package -DskipTests

docker build \
    -t localhost:5001/grocery-inventory-dashboard:3 \
    .
```

The Docker image was successfully created and tagged as:

```
localhost:5001/grocery-inventory-dashboard:3
```

The image was then pushed to the local registry:

```
docker push \
    localhost:5001/grocery-inventory-dashboard:3
```

The push completed successfully with Docker image digest:

```
sha256:1d55fe25516d54bc3e62026cbb7cdb37f724d38c0e2178aaf28401c86641d955
```

The registry verification showed the available image tags:

```
["1","test","3"]
```

Thus, the Jenkins-generated version `3` was successfully stored in the local registry.

5. Automatic Container Deployment

After pushing the image, Jenkins automatically performed the container deployment.

The previous container named `grocery-inventory-cd` was stopped and removed:

```
docker stop grocery-inventory-cd
docker rm grocery-inventory-cd
```

A fresh container was then created from the new versioned image:

```
docker run -d \
    --name grocery-inventory-cd \
    -p 8083:8080 \
    -e SPRING_DATASOURCE_URL="jdbc:mysql://host.docker.internal:3306/groc
    -e SPRING_DATASOURCE_USERNAME="root" \
    -e SPRING_DATASOURCE_PASSWORD="" \
    localhost:5001/grocery-inventory-dashboard:3
```

The running container was verified with:

```
localhost:5001/grocery-inventory-dashboard:3
0.0.0.0:8083 -> 8080/tcp
grocery-inventory-cd
```

The application was made available at:

`http://localhost:8083/automated-grocery-inventory-dashboard/items`

6. Database and Deployment Verification

The deployed container successfully connected to the MySQL database through the Docker host using:

```
jdbc:mysql://host.docker.internal:3306/grocery_inventory
```

The container logs confirmed successful Hikari connection creation, Hibernate initialization, Spring Boot startup, WAR deployment, and Tomcat startup.

The final deployment verification showed the container in the running state and the registry containing the newly generated image tag.

7. End-to-End Pipeline Result

The final Jenkins execution used commit:

```
3df5576
Fix Selenium search stale element handling
```

The testing stage executed all six project tests:

```
Tests run: 6
Failures: 0
Errors: 0
Skipped: 0
BUILD SUCCESS
```

After the successful tests, Jenkins continued automatically through WAR packaging, Docker image creation, registry push, container replacement, and deployment verification.

The final Jenkins result was:

FINISHED: SUCCESS

The pipeline reported:

```
Week 12 Docker deployment completed successfully.
Image: localhost:5001/grocery-inventory-dashboard:3
Application: http://localhost:8083/automated-grocery-inventory-dashboard/
```

8. Evidence

The following screenshots should be included in the final report:

1. Jenkins pipeline overview showing successful stages.
2. Jenkins Test stage showing 6/6 tests passed.
3. Docker image build and versioned tag.
4. Docker Registry showing image tag `3`.
5. Running Docker container and port mapping.
6. Application running at port `8083`.

9. Conclusion

Week 12 successfully implemented continuous deployment for the Automated Grocery Inventory Dashboard.

The Jenkins pipeline now performs testing before deployment and automatically builds a versioned Docker image, publishes it to the local Docker Registry, replaces the previous container, starts a fresh container, and verifies the deployment.

The successful end-to-end execution demonstrated the complete flow from a Git commit to a running Dockerized application.

Week 12 Status: COMPLETED